

AI Assisted Task Manager

Detailed project brief generated from the project record.

CANDIDATE

Rahul Sharma

DURATION

5 days

TECHNOLOGY STACK

JavaScript

TypeScript

React

Next.js

Node.js

Express.js

Tailwind CSS

MongoDB

JWT

Groq API

Project Overview

Summary of the problem, purpose and expected outcome.

PROJECT SUMMARY

A small web app where users can register, manage personal tasks, export a task list as PDF, and receive AI generated task suggestions via the Groq API.

CANDIDATE

Rahul Sharma

TYPE

fullstack

DURATION

5 days

Objectives

- Implement role based JWT authentication.
- Create CRUD endpoints for personal tasks.
- Build a responsive React UI with Tailwind.
- Add PDF export of a user's task list.
- Integrate Groq API to suggest a new task based on existing ones.

Candidate Details

NAME

Rahul Sharma

Technology Stack

Languages, frameworks, databases and tools used for implementation.

JavaScript

TypeScript

React

Next.js

Node.js

Express.js

Tailwind CSS

MongoDB

JWT

Groq API

Functional Requirements

Requirement 1

Users can register and log in; passwords are hashed.

Requirement 2

Authenticated users can create, read, update, and delete their own tasks.

Requirement 3

Task list page shows tasks with status and due date.

Requirement 4

A "Export PDF" button generates a printable PDF of all tasks.

Requirement 5

A "Get AI Suggestion" button calls Groq API and adds a suggested task.

Requirement 6

API returns proper HTTP status codes and validation errors.

Non-Functional Requirements

Requirement 1

Passwords stored with bcrypt hashing.

Requirement 2

JWT tokens must be signed with a secret and expire after 1 hour.

Requirement 3

API response time < 300 ms for CRUD operations.

Requirement 4

UI must be responsive on desktop and mobile.

Requirement 5

Code must include ESLint/Prettier formatting and Git commit history.

System Modules

Major functional areas of the project.

01

Authentication

Handles user registration, login, JWT issuance, and role based access.

- Password hashing
- Login endpoint
- Protected route middleware

02

Task Management

CRUD API and UI for personal tasks.

- Create task
- Read task list
- Update task status

03

AI Suggestion & Export

Integrates Groq API for task suggestions and generates PDF exports.

- Call Groq API
- Add suggested task
- Generate PDF with task data

Implementation Roadmap

Recommended implementation sequence.

PHASE 1

Setup

Create repo, configure Next.js frontend and Express backend, connect to MongoDB.

Deliverables: Git repository with initial commit • README with setup instructions

Tasks: Initialize monorepo with npm workspaces • Add TypeScript config, ESLint, Prettier • Set up MongoDB connection string

PHASE 2

Authentication

Implement secure JWT based auth.

Deliverables: Auth API documentation (Swagger snippet) • Postman collection for auth endpoints

Tasks: Create User model (email, password, role) • Build register & login endpoints • Add auth middleware for protected routes

PHASE 3

Task Management

Provide full CRUD for tasks and UI list.

Deliverables: Live task list page • API test scripts for CRUD

Tasks: Define Task model (title, description, dueDate, status, ownerId) • Implement CRUD endpoints with validation • Build React pages: task list, create/edit form

PHASE 4

AI & PDF

Add AI suggestion button and PDF export feature.

Deliverables: PDF export works for any task list • AI suggestion appears as a new task

Tasks: Create service that calls Groq API with existing tasks • Add "Get AI Suggestion" button to UI • Integrate pdfmake (or similar) to export tasks as PDF

API Reference

Key endpoints exposed by the project.

METHOD	ENDPOINT	DESCRIPTION
POST	/api/auth/register	Create new user account
POST	/api/auth/login	Authenticate and receive JWT
GET	/api/tasks	Retrieve logged in user's tasks
POST	/api/tasks	Create a new task
PUT	/api/tasks/:id	Update task fields
DELETE	/api/tasks/:id	Delete a task

POST	/api/tasks/ai-suggest	Call Groq API and add suggested task
------	-----------------------	--------------------------------------

Evaluation Criteria

Suggested criteria for reviewing implementation.

Criterion 1 All listed functional requirements work as described.
Criterion 2 Consistent TypeScript typing, linting passes, clear commit history.
Criterion 3 Passwords hashed, JWT protected routes, no sensitive data in logs.

Additional Project Details

Testing Plan

- Unit tests for auth service (bcrypt, JWT).
- Integration tests for each CRUD endpoint (using supertest).
- E2E test: register !' login !' create task !' export PDF.
- Mock Groq API call and verify suggestion insertion.
- Lint/format check as part of CI.

Implementation Note This document is generated from the project record. AI-generated requirements should be reviewed by an administrator before implementation.
